The Processor: Part II

CSE 4305: Computer Organization and Architecture

Mohammad Ishrak Abedin
Lecturer, Department of CSE

26 January 2026

Islamic University of Technology

Table of Contents

Pipelined Datapath and Control
Data Hazards: Forwarding vs Stalling
Control Hazards
Conclusion

”

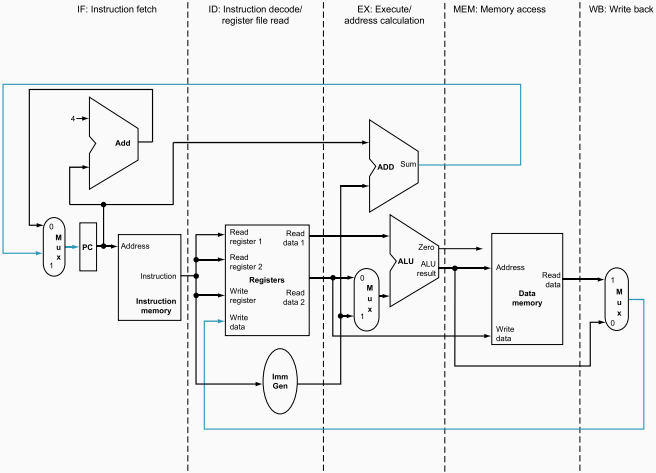
What do you mean, why’s it got to be built? It’s a bypass. You’ve got to build bypasses.

The Hitchhiker’s Guide to the Galaxy, 1979

Douglas Adams

Pipelined Datapath and Control

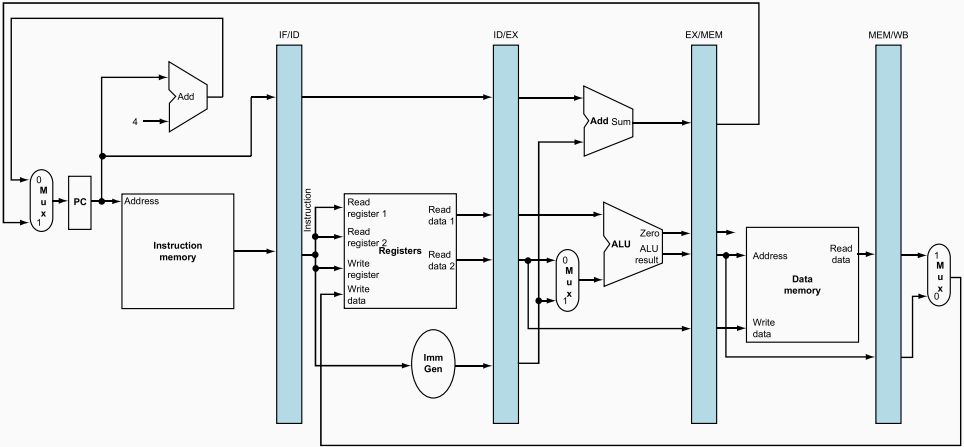
Stages in RISC-V Pipelined Datapath



* Right-to-left flow leads to hazards (WB and MEM stages)

Pipeline Registers

Need registers between stages to hold information produced in the previous cycle



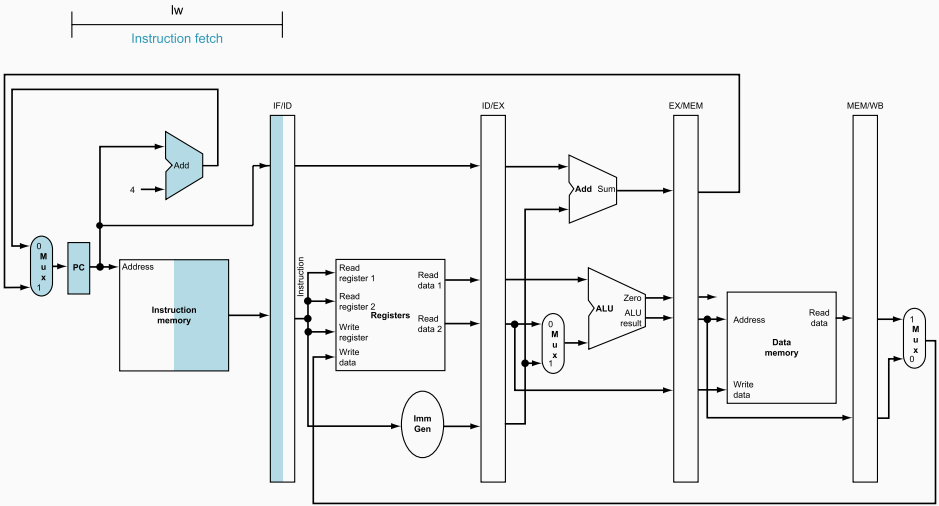
Pipeline Operation

- Cycle-by-cycle flow of instruction through the pipelined datapath
  - Single-clock-cycle pipeline diagram shows pipeline usage in a single cycle, highlighting resources used
  - Multi-clock-cycle diagram depicts graph of operation over time on the contrary

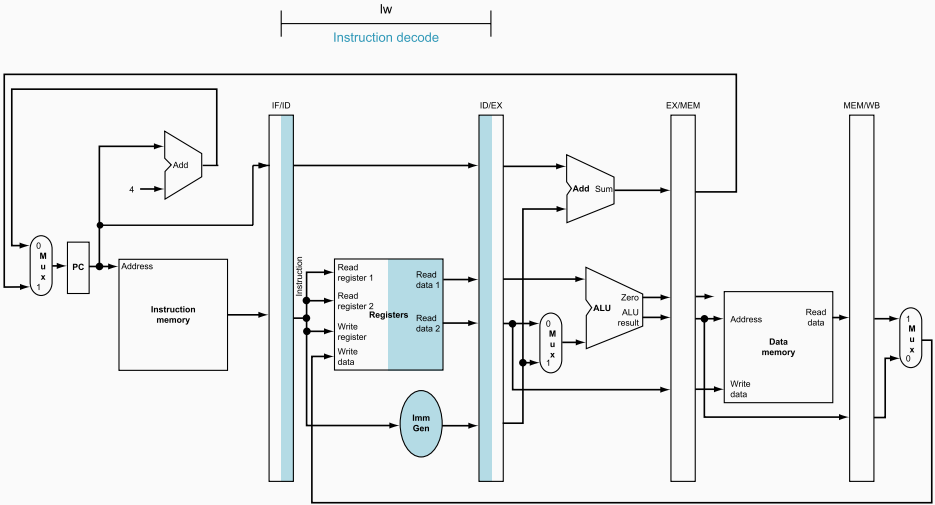
Pipeline Operation

- Cycle-by-cycle flow of instruction through the pipelined datapath
  - Single-clock-cycle pipeline diagram shows pipeline usage in a single cycle, highlighting resources used
  - Multi-clock-cycle diagram depicts graph of operation over time on the contrary
- We will look at the single-clock-cycle diagrams for `lw` and `sw` instructions
  - First two stages (IF and ID) are the same for both `lw` and `sw`

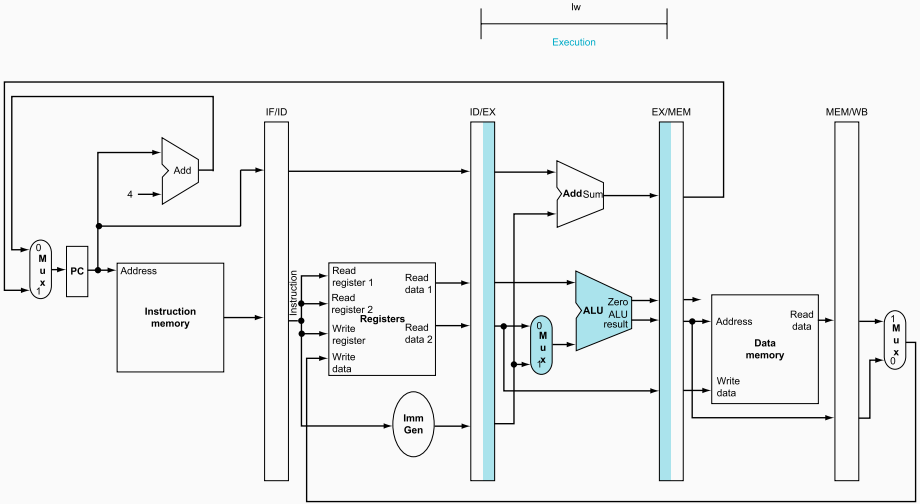
IF for Load


lw
Instruction fetch
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add
Sum
ALU
Zero
ALU result
0
M
u
x
1
Address
Data memory
Read data
Write data
1
M
u
x
0

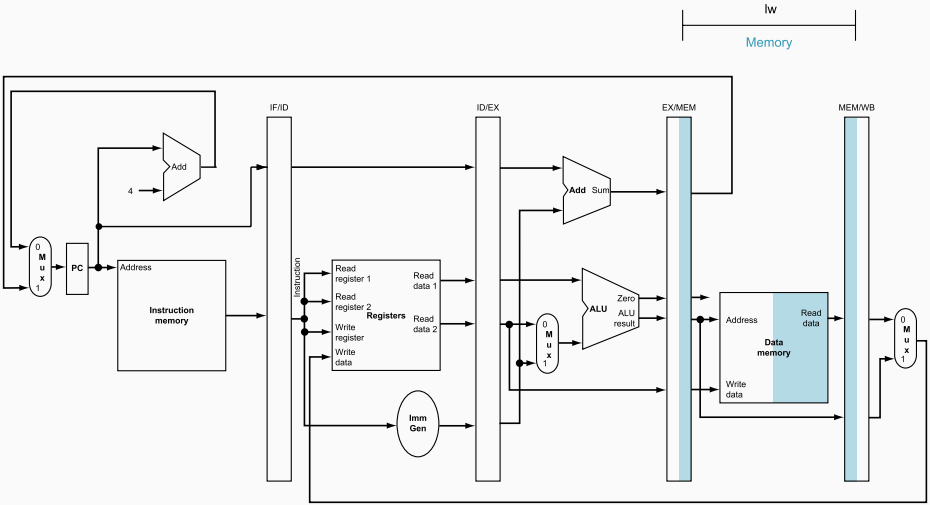
ID for Load


lw
Instruction decode
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add
Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data
1
M
u
x
0

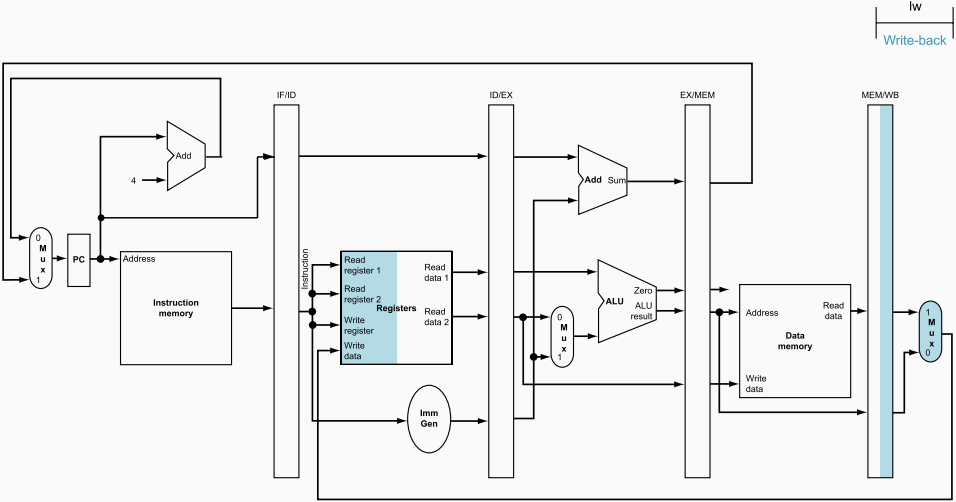
EX for Load


lw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add
Sum
ALU
Zero
ALU result
Address
Read data
Data memory
Write data

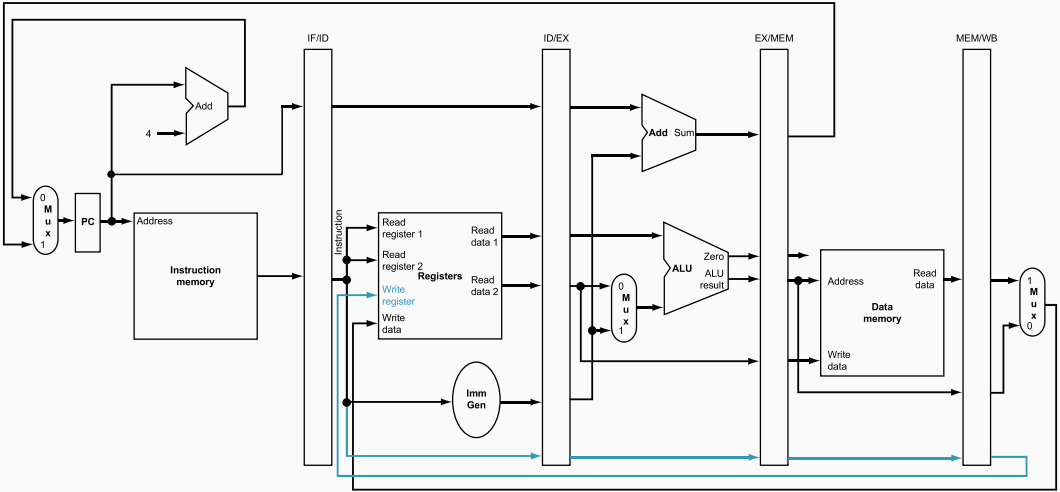
MEM for Load


lw
Memory
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M u x
PC
Address
Instruction memory
instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add
Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

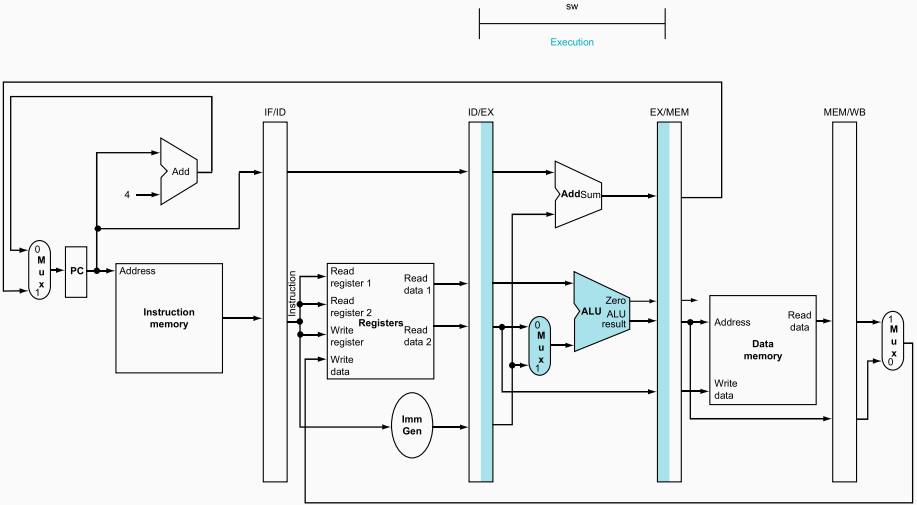
WB for Load (Buggy)



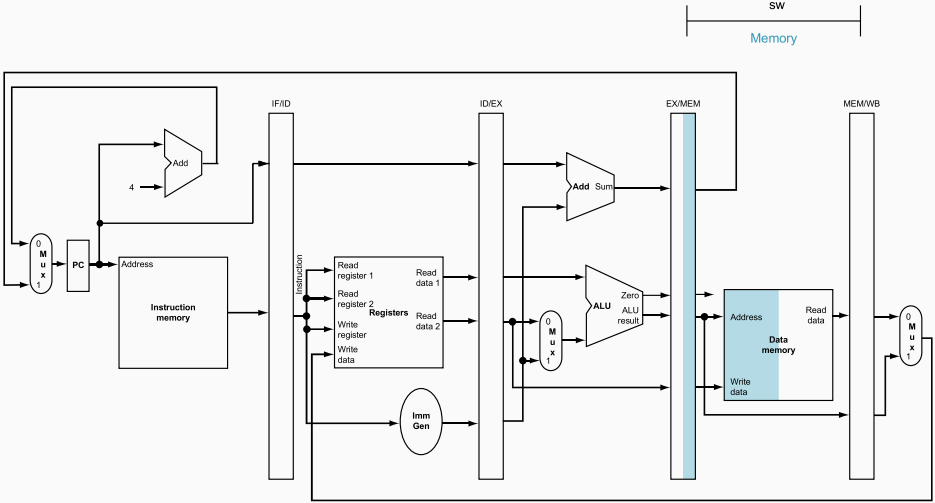
WB for Load (Corrected)



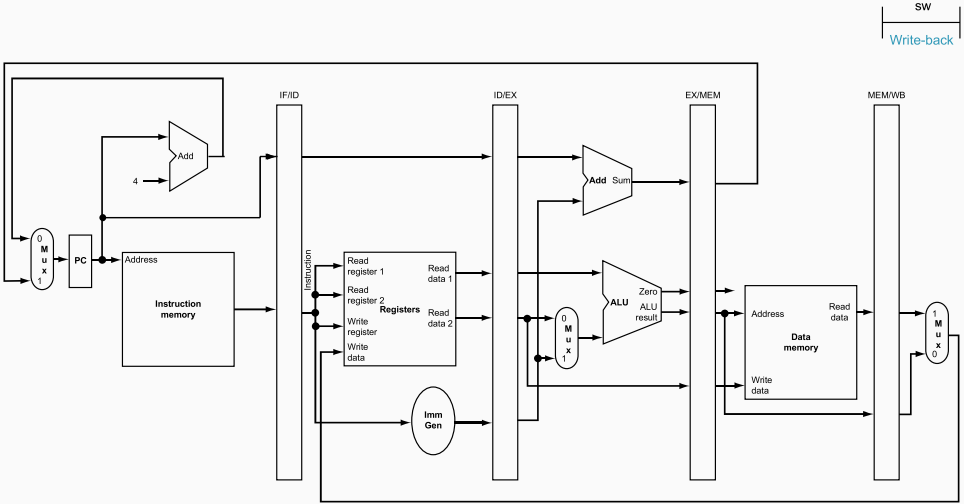
EX for Store


sw
Execution
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
0
M
u
x
1
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
AddSum
ALU
Zero
ALU result
0
M
u
x
1
Address
Read data
Data memory
Write data
1
M
u
x
0

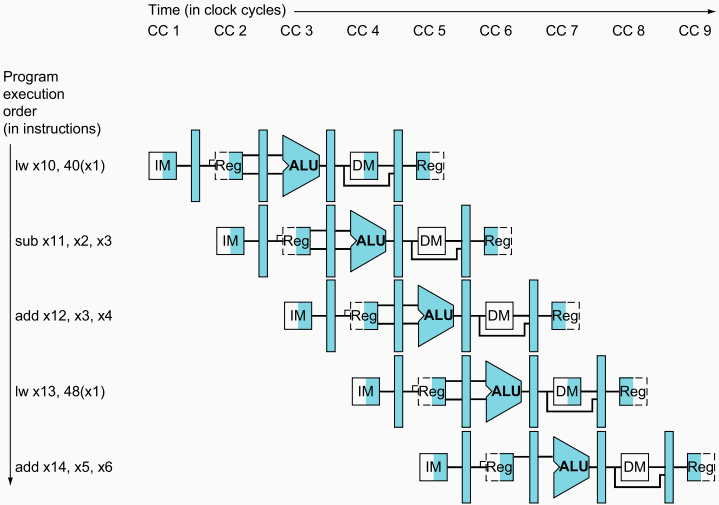
MEM for Store



WB for Store


SW
Write-back
IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
M
u
x
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
Imm Gen
Add
Sum
ALU
Zero
ALU result
Address
Data memory
Read data
Write data

Multiple-Clock-Cycle Pipeline Diagram



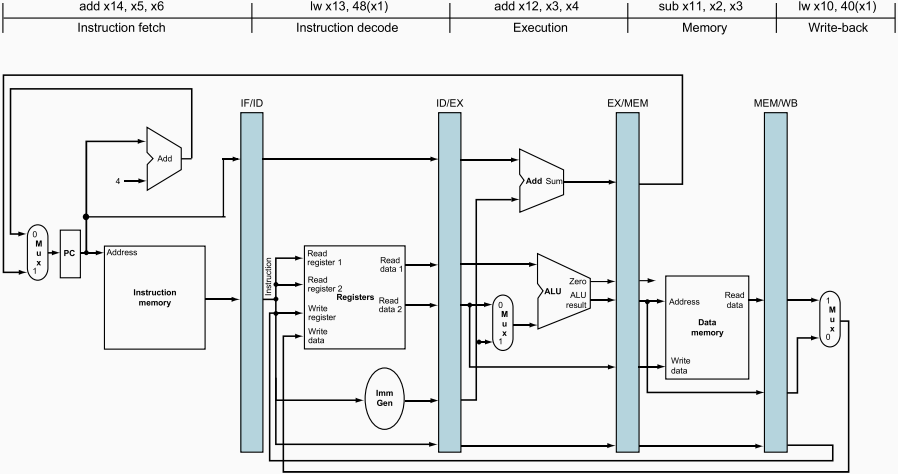
Multiple-Clock-Cycle Pipeline Diagram (Traditional Form)

Time (in clock cycles) →

Program execution order (in instructions) ↓

	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
lw x10, 40(x1)	Instruction fetch	Instruction decode	Execution	Data access	Write-back				
sub x11, x2, x3		Instruction fetch	Instruction decode	Execution	Data access	Write-back			
add x12, x3, x4			Instruction fetch	Instruction decode	Execution	Data access	Write-back		
lw x13, 48(x1)				Instruction fetch	Instruction decode	Execution	Data access	Write-back	
add x14, x5, x6					Instruction fetch	Instruction decode	Execution	Data access	Write-back

Single-Clock-Cycle Pipeline Diagram



Shows the state of a pipeline in a given cycle (vertical slices)

Pipelined Control Overview

- Control for the pipelined datapath borrows heavily from the single-cycle design

Pipelined Control Overview

- Control for the pipelined datapath borrows heavily from the single-cycle design
- We utilize the same ALU control logic, branch logic, and control lines.

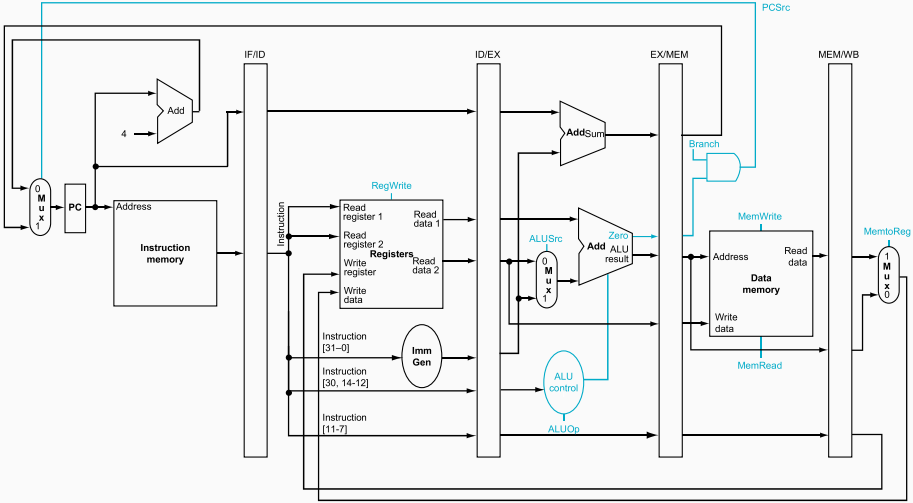
Pipelined Control Overview

- Control for the pipelined datapath borrows heavily from the single-cycle design
- We utilize the same ALU control logic, branch logic, and control lines.
- Key Assumption: The PC and all pipeline registers (IF/ID, ID/EX, EX/MEM, and MEM/WB) are written on every clock cycle.

Pipelined Control Overview

- Control for the pipelined datapath borrows heavily from the single-cycle design
- We utilize the same ALU control logic, branch logic, and control lines.
- Key Assumption: The PC and all pipeline registers (IF/ID, ID/EX, EX/MEM, and MEM/WB) are written on every clock cycle.
- Therefore, no separate write signals are required for these specific state elements.

Pipelined Control Signals (Simplified)



Control Grouping by Stage

Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted

Control Grouping by Stage

Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted
2. *ID:* Source registers are in fixed locations; no special control needed for reading

Control Grouping by Stage

Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted
2. *ID:* Source registers are in fixed locations; no special control needed for reading
3. *EX:* Sets ALUOp and ALUSrc to select the operation and the second ALU operand (Register vs Immediate)

Control Grouping by Stage

Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted
2. *ID:* Source registers are in fixed locations; no special control needed for reading
3. *EX:* Sets ALUOp and ALUSrc to select the operation and the second ALU operand (Register vs Immediate)
4. *MEM:* Sets Branch, MemRead, and MemWrite; PCSrc is determined by the Branch signal combined with the ALU Zero result

Control Grouping by Stage

Control lines are divided into groups according to their active pipeline stage —

1. *IF:* Signals to read memory and write the *PC* are always asserted
2. *ID:* Source registers are in fixed locations; no special control needed for reading
3. *EX:* Sets ALUOp and ALUSrc to select the operation and the second ALU operand (Register vs Immediate)
4. *MEM:* Sets Branch, MemRead, and MemWrite; PCSrc is determined by the Branch signal combined with the ALU Zero result
5. *WB:* Sets MemtoReg to choose between ALU result or memory data and RegWrite to commit the chosen value to the register file

Grouped Pipelined Control Signals

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R-format	10	0	0	0	0	1	0
`lw`	00	1	0	1	0	1	1
`sw`	00	1	0	0	1	0	X
`beq`	01	0	1	0	0	0	X

Implementing Pipelined Control

- Control information is created during the Instruction Decode stage

Implementing Pipelined Control

- Control information is created during the Instruction Decode stage
- Passing Control: Control signals are passed down the pipeline by extending the pipeline registers

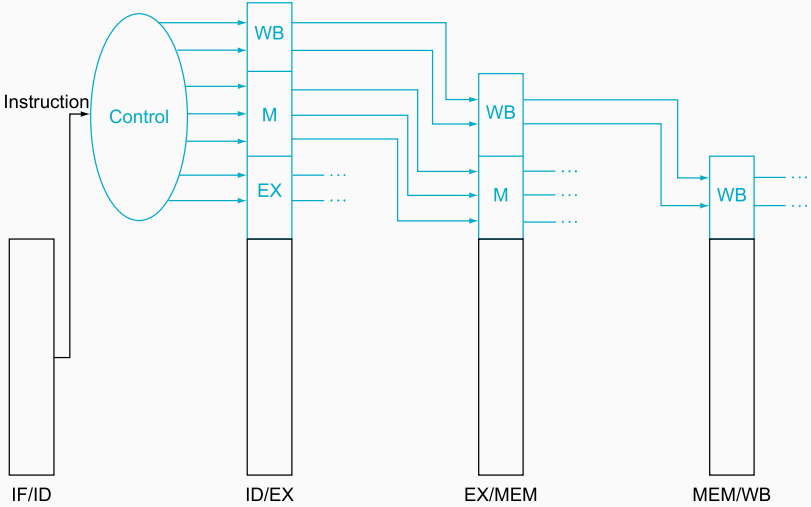
Implementing Pipelined Control

- Control information is created during the Instruction Decode stage
- Passing Control: Control signals are passed down the pipeline by extending the pipeline registers
- As an instruction moves, its backpack of control signals moves with it to the appropriate stage

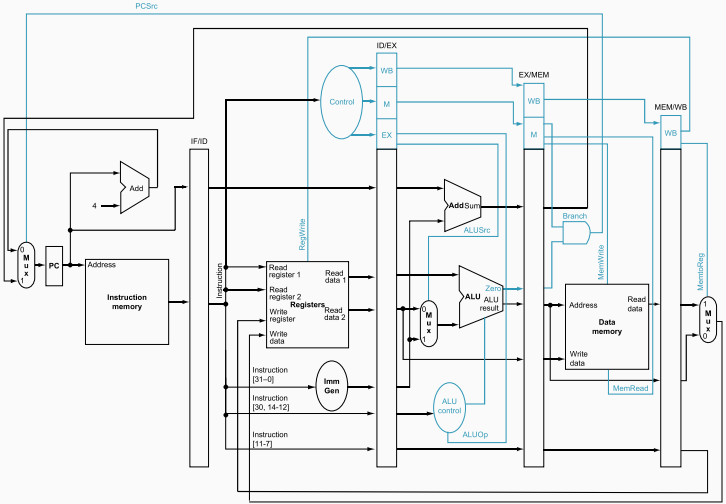
Implementing Pipelined Control

- Control information is created during the Instruction Decode stage
- Passing Control: Control signals are passed down the pipeline by extending the pipeline registers
- As an instruction moves, its backpack of control signals moves with it to the appropriate stage
- This design ensures the signals reach the components at the exact moment the instruction arrives

Pipelined Control Registers and Passing



Pipelined Datapath with Control



Data Hazards

Data Hazards in ALU Instructions

- Consider the following code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

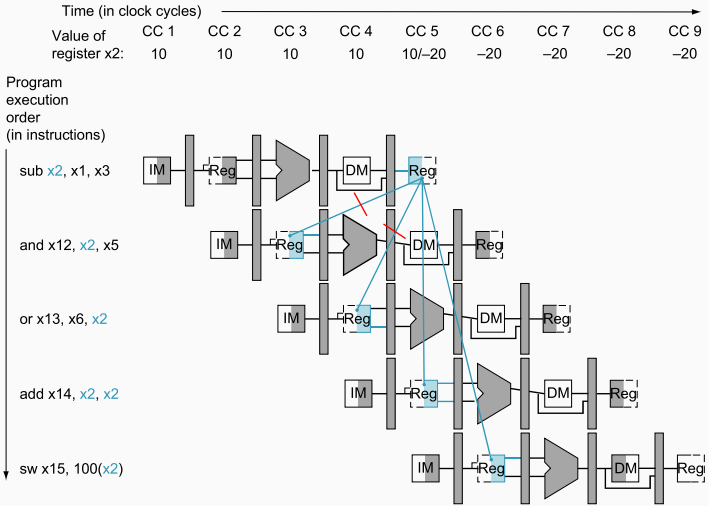
Data Hazards in ALU Instructions

- Consider the following code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- We can resolve the hazards with forwarding, but how do we detect when to forward?

Dependencies and Forwarding



Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register

Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register
- Following this notation, ALU operand register numbers in EX stage are given by ID/EX.RegisterRs1 and ID/EX.RegisterRs2

Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register
- Following this notation, ALU operand register numbers in EX stage are given by ID/EX.RegisterRs1 and ID/EX.RegisterRs2
- We get data hazards when —
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register
- Following this notation, ALU operand register numbers in EX stage are given by ID/EX.RegisterRs1 and ID/EX.RegisterRs2
- We get data hazards when —
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2
- But only if forwarding instruction will write to a register, i.e., EX/MEM.RegWrite or MEM/WB.RegWrite is asserted

Detecting the Need to Forward

- Pass register numbers along pipeline (with values), e.g., ID/EX.RegisterRs1 → register number for Rs1 sitting in the ID/EX pipeline register
- Following this notation, ALU operand register numbers in EX stage are given by ID/EX.RegisterRs1 and ID/EX.RegisterRs2
- We get data hazards when —
  1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
  1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
  2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
  2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2
- But only if forwarding instruction will write to a register, i.e., EX/MEM.RegWrite or MEM/WB.RegWrite is asserted
- And only if Rd for that instruction is not `x0`, i.e., EX/MEM.RegisterRD $\neq 0$, MEM/WB.RegisterRD $\neq 0$

Dependency Detection Example

Example

- For the previous example code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

Dependency Detection Example

Example

- For the previous example code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:

Dependency Detection Example

Example

- For the previous example code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:
  - The `sub-and` is a type 1a hazard (EX/MEM.RegisterRd = ID/EX.RegisterRs1)

Dependency Detection Example

Example

- For the previous example code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:
  - The `sub-and` is a type 1a hazard (EX/MEM.RegisterRd = ID/EX.RegisterRs1)
  - The `sub-or` is a type 2b hazard (MEM/WB.RegisterRd = ID/EX.RegisterRs2)

Dependency Detection Example

Example

- For the previous example code sequence:

```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:
  - The `sub-and` is a type 1a hazard (EX/MEM.RegisterRd = ID/EX.RegisterRs1)
  - The `sub-or` is a type 2b hazard (MEM/WB.RegisterRd = ID/EX.RegisterRs2)
  - The two dependencies on the `sub-add` are not hazards because the register file supplies the proper data during the ID stage of `add`

Dependency Detection Example

Example

- For the previous example code sequence:

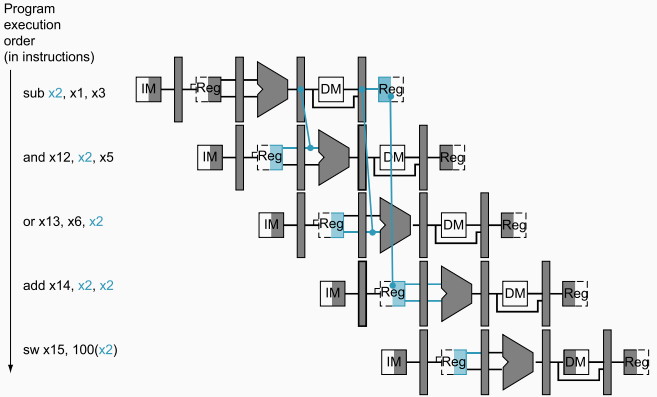
```
sub x2, x1, x3 // Register x2 written by sub
and x12, x2, x5 // 1st operand(x2) depends on sub
or x13, x6, x2 // 2nd operand(x2) depends on sub
add x14, x2, x2 // 1st(x2) & 2nd(x2) depend on sub
sw x15, 100(x2) // Base (x2) depends on sub
```

- Respective hazard conditions are:
  - The `sub-and` is a type 1a hazard (EX/MEM.RegisterRd = ID/EX.RegisterRs1)
  - The `sub-or` is a type 2b hazard (MEM/WB.RegisterRd = ID/EX.RegisterRs2)
  - The two dependencies on the `sub-add` are not hazards because the register file supplies the proper data during the ID stage of `add`
  - There is no data hazard between `sub` and `sw` because `sw` reads `x2` the clock cycle after `sub` writes `x2`

Dependencies after Forwarding

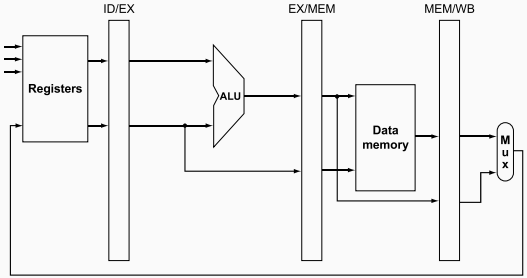
Time (in clock cycles) →

	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6	CC 7	CC 8	CC 9
value of register x2:	10	10	10	10	10/–20	–20	–20	–20	–20



Forwarding Paths

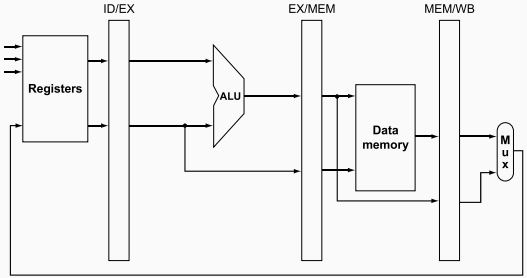
(*The diagrams have been simplified, such as, omission of immediate generation unit)



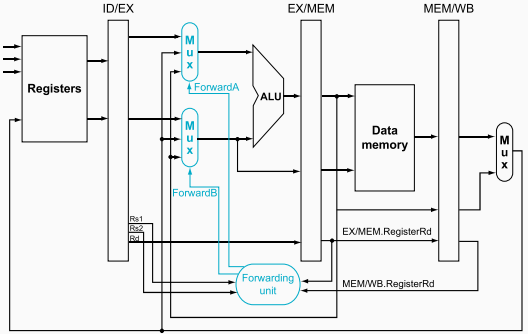
a. No forwarding

Forwarding Paths

(*The diagrams have been simplified, such as, omission of immediate generation unit)



a. No forwarding



b. With forwarding

Forwarding Conditions

Mux Control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result
ForwardA = 01	MEM/WEB	The first ALU operand is forwarded from data memory or an earlier ALU result
ForwardB = 00	ID/EX	The second ALU operand comes from the register file
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result
ForwardB = 01	MEM/WEB	The second ALU operand is forwarded from data memory or an earlier ALU result

Written Conditions for Detecting Hazards (EX Hazards)

```
if  (EX/MEM.RegWrite and
     (EX/MEM.RegisterRd ≠ 0) and
     (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 10
if  (EX/MEM.RegWrite and
     (EX/MEM.RegisterRd ≠ 0) and
     (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 10
```

Written Conditions for Detecting Hazards (MEM Hazards)

```
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 01
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 01
```

Double Data Hazard

- Consider the following code sequence:

```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```

Double Data Hazard

- Consider the following code sequence:

```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```

- Both hazards occur; we want to use the most recent

Double Data Hazard

- Consider the following code sequence:

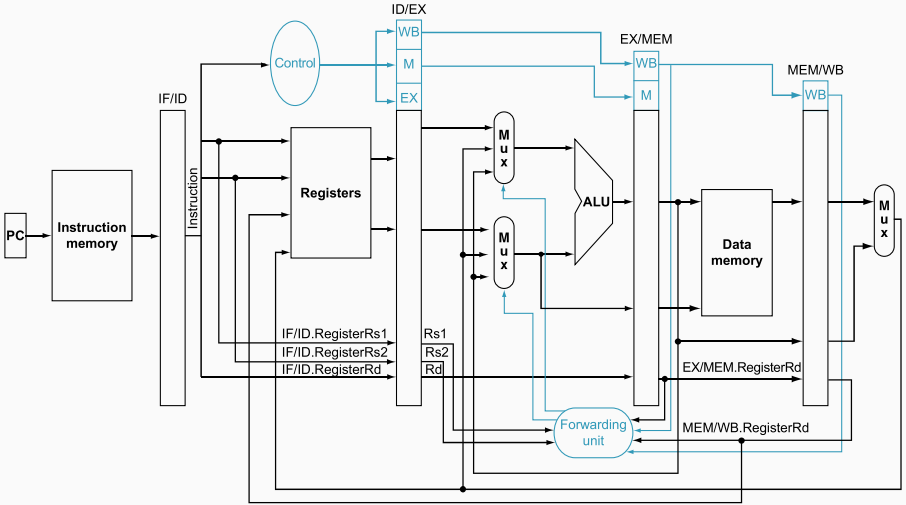
```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```

- Both hazards occur; we want to use the most recent
- Revise MEM hazard condition to only forward if EX hazard condition is not true

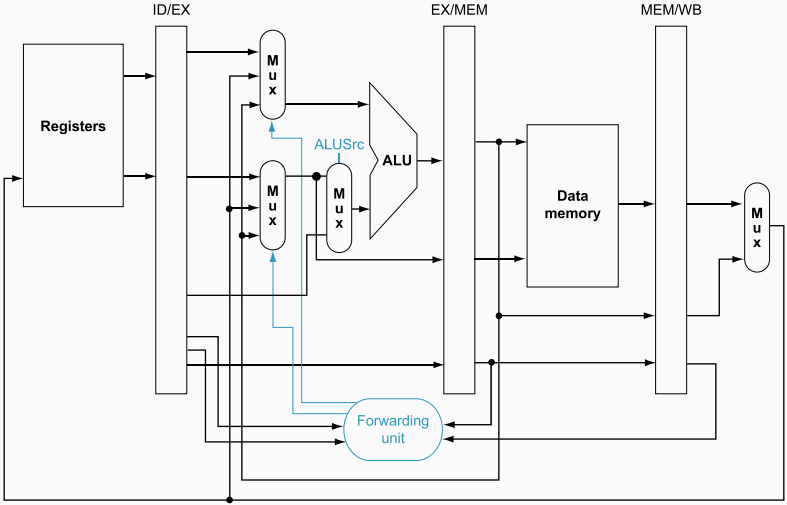
Revised MEM Hazard Forwarding Condition

```
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
         and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
         ForwardA = 01
if  (MEM/WB.RegWrite and
     (MEM/WB.RegisterRd ≠ 0) and
     not(EX/MEM.RegWrite and (EX/MEM.RegisterRd ≠ 0)
         and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) and
     (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
         ForwardB = 01
```

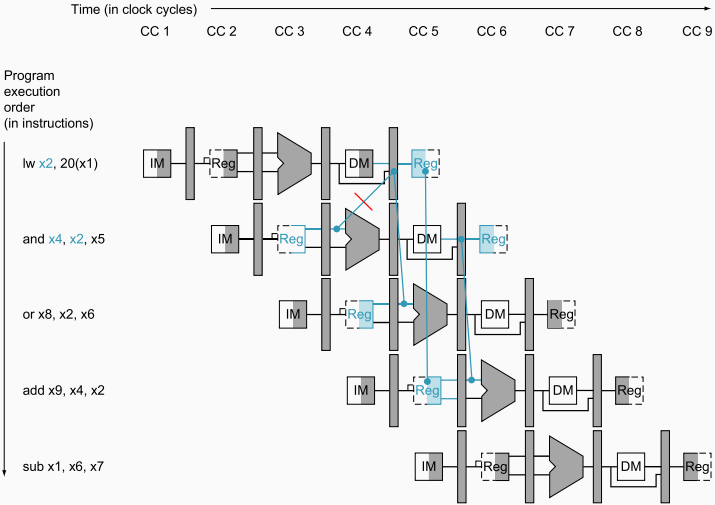
Datapath with Forwarding



Forwarding and Signed Immediate with a 2:1 Multiplexor



Load-Use Hazard in the Pipeline



Load-Use Hazard Detection

- Check when the instruction is decoded in the ID stage

Load-Use Hazard Detection

- Check when the instruction is decoded in the ID stage
- ALU operand register numbers in the ID stage are given by IF/ID.RegisterRs1 and IF/ID.RegisterRs2

Load-Use Hazard Detection

- Check when the instruction is decoded in the ID stage
- ALU operand register numbers in the ID stage are given by IF/ID.RegisterRs1 and IF/ID.RegisterRs2
- Load-use hazard condition:

```
if  (ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
        (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    // stall the pipeline
```

Load-Use Hazard Detection

- Check when the instruction is decoded in the ID stage
- ALU operand register numbers in the ID stage are given by IF/ID.RegisterRs1 and IF/ID.RegisterRs2
- Load-use hazard condition:

```
if  (ID/EX.MemRead and
    ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
        (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    // stall the pipeline
```

- If detected, stall and insert bubble

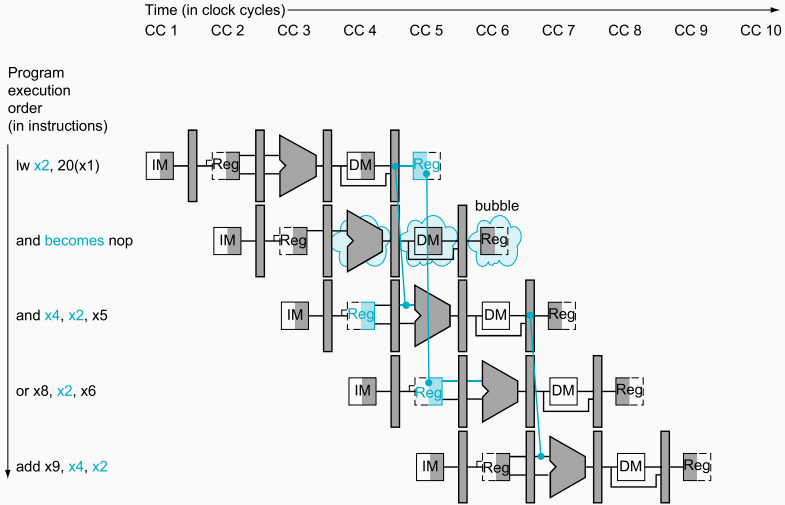
How to Stall the Pipeline

- Force control values in the ID/EX register to 0 (zero)
  - EX, MEM, and WB do nop (no-operation)

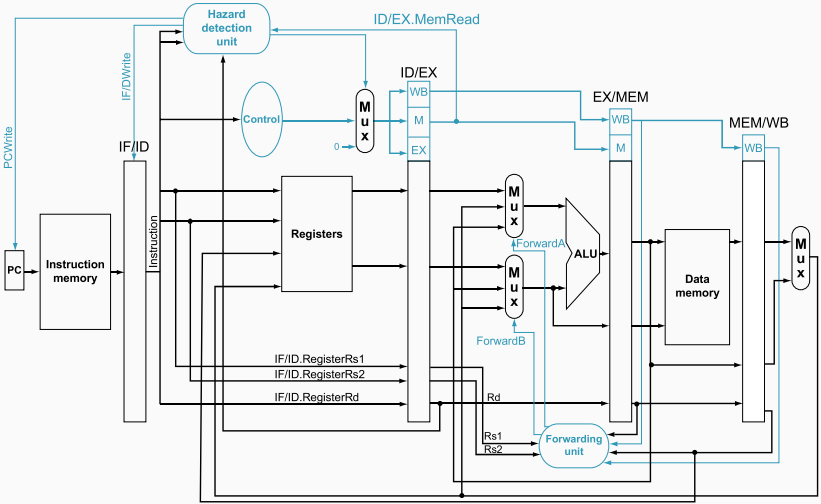
How to Stall the Pipeline

- Force control values in the ID/EX register to 0 (zero)
  - EX, MEM, and WB do nop (no-operation)
- Prevent the update of *PC* and IF/ID register
  - Used instruction is decoded again
  - Following instruction is fetched again
  - 1-cycle stall allows MEM to read data for `lw`
  - Can subsequently forward to the EX stage

Stalling for Load-Use Data Hazard



Datapath with Hazard Detection and Forwarding



Stall and Performance

The BIG Picture

- Stalls reduce performance
  - But are required to get correct results

Stall and Performance

The BIG Picture

- Stalls reduce performance
  - But are required to get correct results
- Compilers can arrange code to avoid hazards and stalls
  - Require knowledge of the pipeline structure

Control Hazards

Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards

Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards
- An instruction must be fetched every clock cycle to sustain the pipeline

Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards
- An instruction must be fetched every clock cycle to sustain the pipeline
- In standard design, branch decision occurs in the MEM pipeline stage

Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards
- An instruction must be fetched every clock cycle to sustain the pipeline
- In standard design, branch decision occurs in the MEM pipeline stage
- Delay in determining the proper instruction to fetch is a control hazard (or branch hazard).

Control Hazards Overview

- Hazards involving conditional branches are distinct from data hazards
- An instruction must be fetched every clock cycle to sustain the pipeline
- In standard design, branch decision occurs in the MEM pipeline stage
- Delay in determining the proper instruction to fetch is a control hazard (or branch hazard).
- Comparison to data hazards:
  - Simpler to understand
  - Occur less frequently
  - Forwarding is not directly effective against them

Scheme to Resolve Control Hazards: Assume Branch Not Taken

- Stalling until branch completion is too slow

Scheme to Resolve Control Hazards: Assume Branch Not Taken

- Stalling until branch completion is too slow
- Optimization strategy:
  - Predict conditional branch will not be taken
  - Continue execution down sequential stream

Scheme to Resolve Control Hazards: Assume Branch Not Taken

- Stalling until branch completion is too slow
- Optimization strategy:
  - Predict conditional branch will not be taken
  - Continue execution down sequential stream
- If prediction is correct: No cost

Scheme to Resolve Control Hazards: Assume Branch Not Taken

- Stalling until branch completion is too slow
- Optimization strategy:
  - Predict conditional branch will not be taken
  - Continue execution down sequential stream
- If prediction is correct: No cost
- If prediction is wrong (branch taken):
  - Fetched/decoded instructions must be discarded
  - Execution continues at branch target

Scheme to Resolve Control Hazards: Assume Branch Not Taken

- Stalling until branch completion is too slow
- Optimization strategy:
  - Predict conditional branch will not be taken
  - Continue execution down sequential stream
- If prediction is correct: No cost
- If prediction is wrong (branch taken):
  - Fetched/decoded instructions must be discarded
  - Execution continues at branch target
- Halves control hazard cost if branches are untaken 50% of the time.

Discarding Instructions (Flushing)

- Discarding involves changing original control values to 0s

Discarding Instructions (Flushing)

- Discarding involves changing original control values to 0s
- Similar to stalling for load-use data hazards, but more extensive

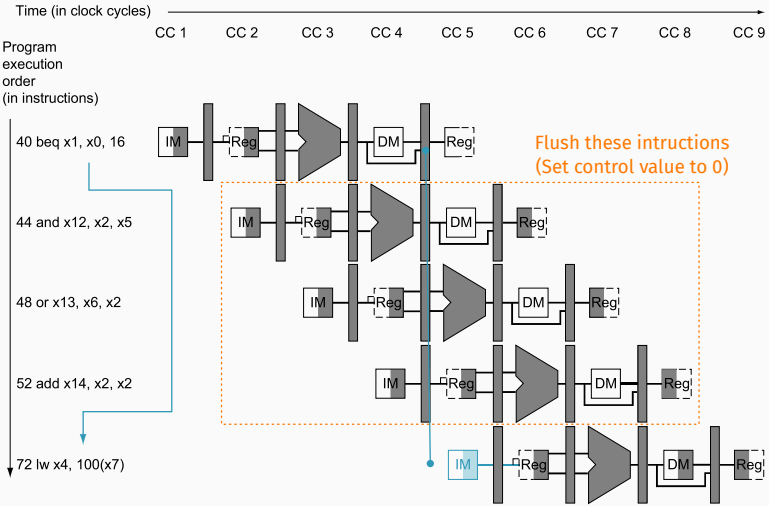
Discarding Instructions (Flushing)

- Discarding involves changing original control values to 0s
- Similar to stalling for load-use data hazards, but more extensive
- When branch reaches MEM stage, must flush instructions in:
  1. IF stage
  2. ID stage
  3. EX stage

Discarding Instructions (Flushing)

- Discarding involves changing original control values to 0s
- Similar to stalling for load-use data hazards, but more extensive
- When branch reaches MEM stage, must flush instructions in:
  1. IF stage
  2. ID stage
  3. EX stage
- In this case, the cost is delay by three stages

Example Multi-Cycle Diagram of Control Hazard



Optimization: Reducing Branch Delay

- Goal: Reduce cost of a taken branch

Optimization: Reducing Branch Delay

- Goal: Reduce cost of a taken branch
- Method: Move conditional branch execution earlier in pipeline

Optimization: Reducing Branch Delay

- Goal: Reduce cost of a taken branch
- Method: Move conditional branch execution earlier in pipeline
- Move from MEM stage to ID stage

Optimization: Reducing Branch Delay

- Goal: Reduce cost of a taken branch
- Method: Move conditional branch execution earlier in pipeline
- Move from MEM stage to ID stage
- Required actions in ID:
  - Compute branch target address
  - Evaluate branch decision

Moving Logic to ID Stage

- Address Calculation:
  - Easy to move
  - Move branch adder from EX to ID
  - Uses PC value and immediate field from IF/ID register

Moving Logic to ID Stage

- Address Calculation:
  - Easy to move
  - Move branch adder from EX to ID
  - Uses PC value and immediate field from IF/ID register
- Branch Decision:
  - Harder to implement (dependency)
  - Equality test: XOR individual bit positions of two registers and OR the resultant bits

Moving Logic to ID Stage

- Address Calculation:
  - Easy to move
  - Move branch adder from EX to ID
  - Uses PC value and immediate field from IF/ID register
- Branch Decision:
  - Harder to implement (dependency)
  - Equality test: XOR individual bit positions of two registers and OR the resultant bits
  - Zero output means registers are equal

Complications: Forwarding and Hazards

- Moving branch test to ID implies additional hardware

Complications: Forwarding and Hazards

- Moving branch test to ID implies additional hardware
- Complication 1: Forwarding
  - Must decode, decide bypass, and test equality within ID
  - Requires new forwarding logic (distinct from ALU forwarding)
  - Operands may come from EX/MEM or MEM/WB registers

Complications: Forwarding and Hazards

- Moving branch test to ID implies additional hardware
- Complication 1: Forwarding
  - Must decode, decide bypass, and test equality within ID
  - Requires new forwarding logic (distinct from ALU forwarding)
  - Operands may come from EX/MEM or MEM/WB registers
- Complication 2: Data Hazards
  - ALU instruction immediately preceding branch: Requires 1 stall
  - Load immediately preceding branch: Requires 2 stalls

Implementation and Results

- Moving execution to ID reduces taken branch penalty to one instruction (the one currently being fetched).

Implementation and Results

- Moving execution to ID reduces taken branch penalty to one instruction (the one currently being fetched).
- Flushing mechanism:
  - Add control line IF.Flush
  - Make zero the instruction field of IF/ID pipeline register (instruction becomes `add x0, x0, x0`)
  - Transform fetched instruction into a **nop** (no operation)

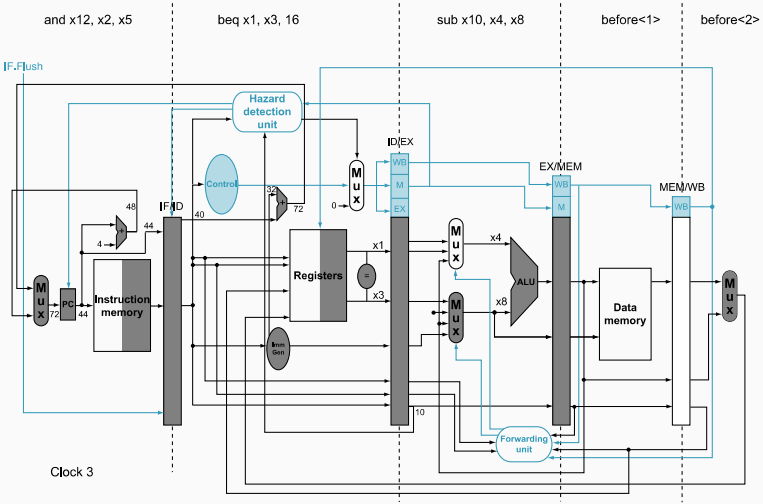
Example: Pipelined Branch

Consider the following RISC-V code with:

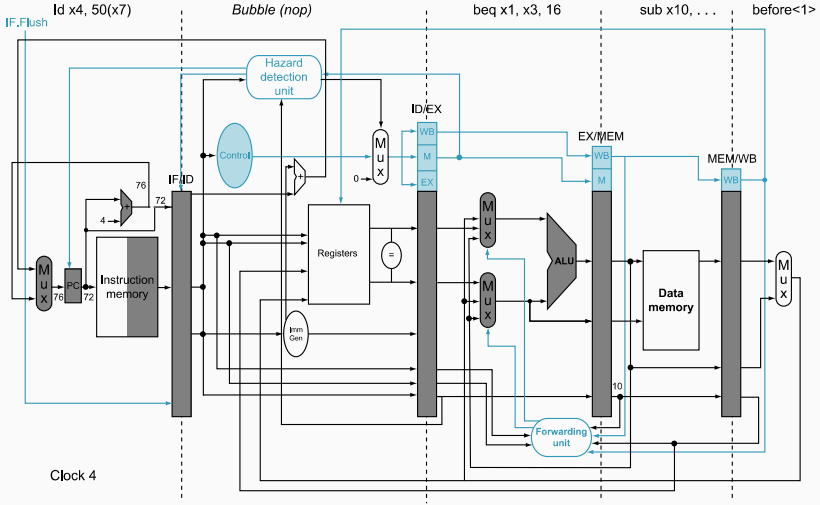
- Branch execution moved to ID, optimized for not taken
- If branch is taken, only one pipeline bubble occurs

```
36 sub x10, x4, x8
40 beq  x1, x3, 16 // PC-relative branch to 40+16*2=72
44 and x12, x2, x5
48 or  x13, x2, x6
52 add x14, x4, x2
56 sub x15, x6, x7
. . .
72 lw   x4, 50(x7)
```

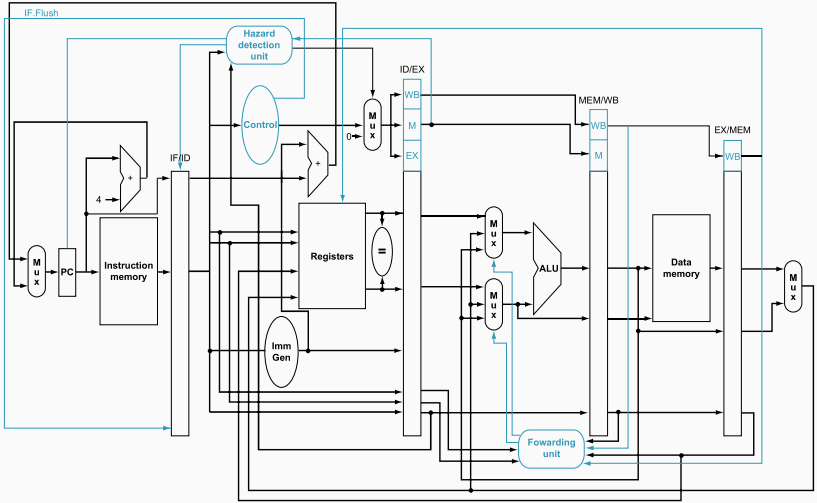
Example: Branch Taken



Example: Branch Taken



Final Datapath and Control



Dynamic Branch Prediction

- Static prediction (Assume Not Taken) is adequate for simple pipelines

Dynamic Branch Prediction

- Static prediction (Assume Not Taken) is adequate for simple pipelines
- Performance issues arise with:
  - Deeper pipelines: Branch penalty increases in clock cycles
  - Multiple-issue pipelines: Branch penalty increases in instructions lost

Dynamic Branch Prediction

- Static prediction (Assume Not Taken) is adequate for simple pipelines
- Performance issues arise with:
  - Deeper pipelines: Branch penalty increases in clock cycles
  - Multiple-issue pipelines: Branch penalty increases in instructions lost
- Aggressive pipelines require dynamic branch prediction

Dynamic Branch Prediction

- Static prediction (Assume Not Taken) is adequate for simple pipelines
- Performance issues arise with:
  - Deeper pipelines: Branch penalty increases in clock cycles
  - Multiple-issue pipelines: Branch penalty increases in instructions lost
- Aggressive pipelines require dynamic branch prediction
- Concept: Look up instruction address to check past behavior

Branch Prediction Buffer (1-Bit Scheme)

- Also called a branch history table

Branch Prediction Buffer (1-Bit Scheme)

- Also called a branch history table
- Small memory indexed by lower portion of branch instruction address

Branch Prediction Buffer (1-Bit Scheme)

- Also called a branch history table
- Small memory indexed by lower portion of branch instruction address
- Contains a bit indicating if branch was recently taken

Branch Prediction Buffer (1-Bit Scheme)

- Also called a branch history table
- Small memory indexed by lower portion of branch instruction address
- Contains a bit indicating if branch was recently taken
- Mechanism:
  - Uses prediction as a hint for fetching
  - When hint is wrong: delete instructions, invert prediction bit, fetch proper sequence

Shortcoming of 1-Bit Prediction

- Performance issue – can mispredict twice for a branch that is almost always taken

Shortcoming of 1-Bit Prediction

- Performance issue – can mispredict twice for a branch that is almost always taken
- Example Scenario: Nested loop, inner loop branches 9 times, not taken once (exiting)

Shortcoming of 1-Bit Prediction

- Performance issue – can mispredict twice for a branch that is almost always taken
- Example Scenario: Nested loop, inner loop branches 9 times, not taken once (exiting)
- Mispredictions:
  - Last iteration: Inevitable (predicts taken, actually not taken).
  - First iteration: Bit was flipped on previous exit (predicts not taken, actually taken).

Shortcoming of 1-Bit Prediction

- Performance issue – can mispredict twice for a branch that is almost always taken
- Example Scenario: Nested loop, inner loop branches 9 times, not taken once (exiting)
- Mispredictions:
  - Last iteration: Inevitable (predicts taken, actually not taken).
  - First iteration: Bit was flipped on previous exit (predicts not taken, actually taken).
- Result: 80% accuracy for a branch taken 90% of the time.

2-Bit Prediction Scheme

- Solution to the 1-bit oscillation problem

2-Bit Prediction Scheme

- Solution to the 1-bit oscillation problem
- Prediction must be wrong twice before it is changed

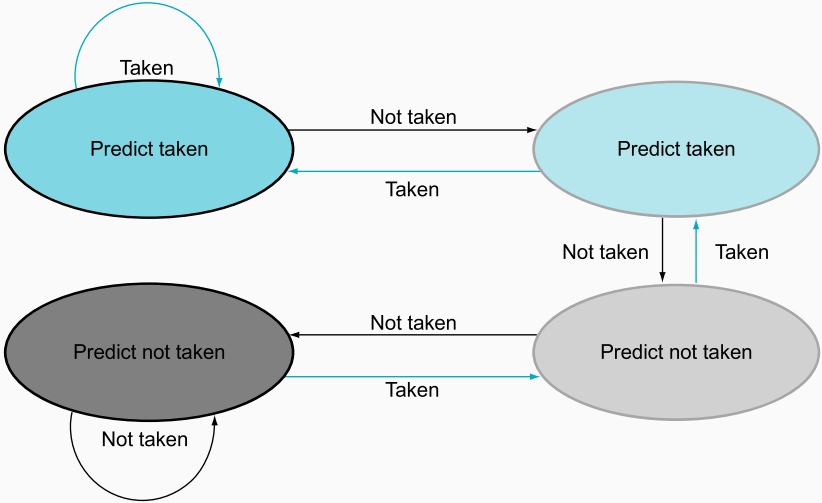
2-Bit Prediction Scheme

- Solution to the 1-bit oscillation problem
- Prediction must be wrong twice before it is changed
- Implementation:
  - Finite-state machine
  - Accessed with instruction address during IF stage

2-Bit Prediction Scheme

- Solution to the 1-bit oscillation problem
- Prediction must be wrong twice before it is changed
- Implementation:
  - Finite-state machine
  - Accessed with instruction address during IF stage
- If predicted taken, fetching begins as soon as PC is known (potentially ID stage)

The States and Transitions in a 2-Bit Prediction Scheme



Branch Target Buffer

- Prediction tells *direction*, but target address calculation still required

Branch Target Buffer

- Prediction tells *direction*, but target address calculation still required
- Calculation takes one cycle in 5-stage pipeline (one-cycle penalty)

Branch Target Buffer

- Prediction tells *direction*, but target address calculation still required
- Calculation takes one cycle in 5-stage pipeline (one-cycle penalty)
- Optimization: Use a Branch Target Buffer

Branch Target Buffer

- Prediction tells *direction*, but target address calculation still required
- Calculation takes one cycle in 5-stage pipeline (one-cycle penalty)
- Optimization: Use a Branch Target Buffer
- Acts as a cache to hold:
  - Destination program counter, or
  - Destination instruction

Correlating Predictors

- 2-bit scheme only uses local branch information

Correlating Predictors

- 2-bit scheme only uses local branch information
- Correlating Predictors: Combine local branch info with global behavior of recently executed branches

Correlating Predictors

- 2-bit scheme only uses local branch information
- Correlating Predictors: Combine local branch info with global behavior of recently executed branches
- Implementation:
  - Two 2-bit predictors per branch
  - Selection based on whether last executed branch was taken/not taken

Correlating Predictors

- 2-bit scheme only uses local branch information
- Correlating Predictors: Combine local branch info with global behavior of recently executed branches
- Implementation:
  - Two 2-bit predictors per branch
  - Selection based on whether last executed branch was taken/not taken
- Global behavior acts as additional index-bits

Tournament Predictors

- Use multiple predictors and track the best performer for each branch

Tournament Predictors

- Use multiple predictors and track the best performer for each branch
- Typical structure:
  - One predictor based on local information
  - One predictor based on global behavior

Tournament Predictors

- Use multiple predictors and track the best performer for each branch
- Typical structure:
  - One predictor based on local information
  - One predictor based on global behavior
- Selector:
  - Chooses which predictor to use
  - Operates like a counter, favoring the more accurate predictor

Tournament Predictors

- Use multiple predictors and track the best performer for each branch
- Typical structure:
  - One predictor based on local information
  - One predictor based on global behavior
- Selector:
  - Chooses which predictor to use
  - Operates like a counter, favoring the more accurate predictor
- Common in recent microprocessors.

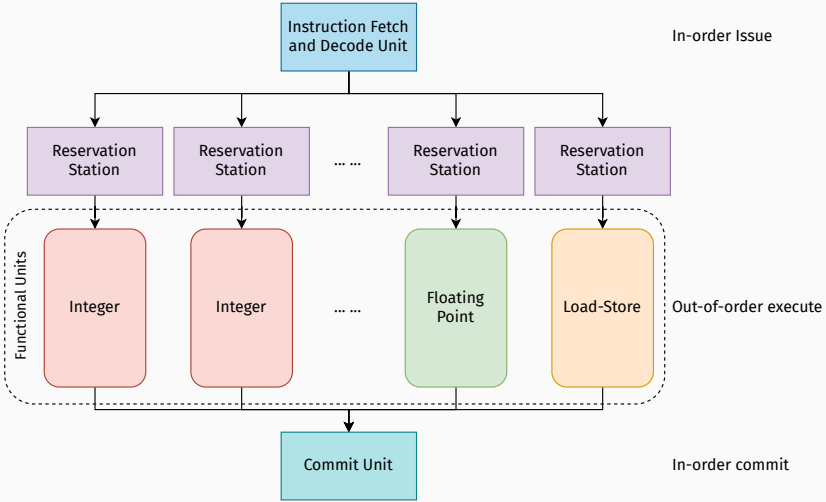
Conclusion

Topics Left Out

The following topics were not covered as deemed advanced for the current level:

- Exceptions
- Concept of Speculation
- Multiple-Issue (Instruction Level Parallelism (ILP))
  - Static Multiple-Issue (VLIW)
  - Dynamic Multiple-Issue (Superscalars and OoO)
- Energy Efficiency and Advanced Pipelining

Three Primary Units of a Dynamically Scheduled Pipelined



Fallacies

- Pipelining is easy!!!
  - The basic idea is easy
  - The devil is in the details, e.g., detecting data hazards

Fallacies

- Pipelining is easy!!!
  - The basic idea is easy
  - The devil is in the details, e.g., detecting data hazards
- Pipelining is independent of technology
  - So why have we not always done pipelining?
  - More transistors make more advanced techniques feasible
  - Pipeline-related ISA design needs to take into account the technology trends, e.g., predicted instructions

Pitfalls

Poor ISA design can make pipelining harder

Pitfalls

Poor ISA design can make pipelining harder

- Complex instruction sets (VAX, IA-32)
  - Significant overhead to make pipelining work
  - IA-32 micro-op approach

Pitfalls

Poor ISA design can make pipelining harder

- Complex instruction sets (VAX, IA-32)
  - Significant overhead to make pipelining work
  - IA-32 micro-op approach
- Complex addressing modes
  - Register update side effects, memory indirection

Pitfalls

Poor ISA design can make pipelining harder

- Complex instruction sets (VAX, IA-32)
  - Significant overhead to make pipelining work
  - IA-32 micro-op approach
- Complex addressing modes
  - Register update side effects, memory indirection
- Delayed branches
  - Advanced pipelines have long delay slots

Concluding Remarks

- ISA influences design of datapath and control

Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA

Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced

Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, and control

Concluding Remarks

- ISA influences design of datapath and control
- Datapath and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
  - More instructions completed per second
  - Latency for each instruction not reduced
- Hazards: structural, data, and control
- Multiple-issue and dynamic scheduling (ILP)
  - Dependencies limit achievable parallelism
  - Complexity leads to the power wall

Thank You!